

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №3.

Тема: «Перегрузка операторов.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Создать класс Money для работы с денежными суммами. Число должно быть представлено двумя полями: типа long для рублей и типа int для копеек. Дробная часть числа при выводе на экран должна быть отделена от целой части запятой. Реализовать:

- операции сравнения (==, !=).
- вычитание копеек (-) (постфиксная и префиксная формы).

Анализ задачи

1. Создание объектов. Сначала создаётся m1 через конструктор без параметров, оба поля обнуляются. Затем m2 через конструктор с параметрами 150 и 75, поля заполняются, вызывается normalize, но так как 75 в пределах нормы, ничего не меняется. Затем m3 через конструктор копирования от m2, поля просто копируются.
2. Работа сеттеров. У m1 вызывают setRubles(10), поле становится 10, нормализация ничего не делает. Затем setKopecks(50), поле становится 50. Затем setMoney(5,150), поля становятся 5 и 150, normalize видит что копеек больше ста, переводит лишнее в рубли, получается 6 рублей 50 копеек.
3. Оператор присваивания. m4 = m2, проверяется что это не сам объект, поля копируются, m4 становится равным 150 рублей 75 копеек.
4. Сравнение. a(100,50) и b(100,50) сравниваются через ==, оба поля совпадают, результат истина. a и c(200,0) сравниваются через !=, оператор инвертирует результат ==, так как рубли разные, == даёт ложь, а != даёт истину.
5. Префиксный декремент. d(10,0), вызывается --d. Копейки равны нулю, рубли больше нуля, поэтому рубли становятся 9, копейки 99. Возвращается ссылка на изменённый объект.
6. Постфиксный декремент. e(5,30), вызывается e--. Создаётся копия с текущими значениями, затем для e вызывается префиксный декремент, копейки становятся 29, возвращается копия со старыми значениями 5,30.
7. Граничные случаи. zero(0,0) после --zero остаётся нулём. ruble(1,0) после --ruble становится 0,99. test(0,99) после --test становится 0,98.

Код

```
#include <iostream>
#include <iomanip>
#include <cmath>
```

```

using namespace std;

class Money {
private:
    long rubles;    // рубли
    int kopecks;    // копейки (0-99)

    // Вспомогательная функция для нормализации копеек
    void normalize() {
        if (kopecks >= 100) {
            rubles += kopecks / 100;
            kopecks %= 100;
        }
        else if (kopecks < 0) {
            long rubles_to_subtract = (abs(kopecks) + 99) / 100;
            rubles -= rubles_to_subtract;
            kopecks += rubles_to_subtract * 100;
        }
    }

public:
    // Конструктор без параметров
    Money() : rubles(0), kopecks(0) {}

    // Конструктор с параметрами
    Money(long r, int k) : rubles(r), kopecks(k) {
        normalize();
    }

    // Конструктор копирования
    Money(const Money& other) : rubles(other.rubles), kopecks(other.kopecks) {}

    // Деструктор
    ~Money() {}

    // Геттеры
    long getRubles() const { return rubles; }
    int getKopecks() const { return kopecks; }

    // Сеттеры
    void setRubles(long r) {
        rubles = r;
        normalize();
    }

    void setKopecks(int k) {
        kopecks = k;
        normalize();
    }

    void setMoney(long r, int k) {
        rubles = r;
        kopecks = k;
        normalize();
    }

    // Оператор присваивания
    Money& operator=(const Money& other) {
        if (this != &other) {
            rubles = other.rubles;
            kopecks = other.kopecks;
        }
        return *this;
    }

    // Операции сравнения
    bool operator==(const Money& other) const {
        return (rubles == other.rubles && kopecks == other.kopecks);
    }
}

```

```

bool operator!=(const Money& other) const {
    return !(*this == other);
}

// Префиксный декремент (--Money)
Money& operator--() {
    if (kopecks == 0) {
        if (rubles > 0) {
            rubles--;
            kopecks = 99;
        }
        else {
            rubles = 0;
            kopecks = 0;
        }
    }
    else {
        kopecks--;
    }
    return *this;
}

// Постфиксный декремент (Money--)
Money operator--(int) {
    Money temp = *this;
    --(*this);
    return temp;
}

// Дружественные функции для ввода/вывода
friend ostream& operator<<(ostream& os, const Money& m);
friend istream& operator>>(istream& is, Money& m);
};

// Оператор вывода
ostream& operator<<(ostream& os, const Money& m) {
    os << m.rubles << ",";
    if (m.kopecks < 10) {
        os << "0" << m.kopecks;
    }
    else {
        os << m.kopecks;
    }
    return os;
}

// Оператор ввода
istream& operator>>(istream& is, Money& m) {
    char comma;
    long r;
    int k;

    is >> r >> comma >> k;

    if (comma == ',') {
        m.rubles = r;
        m.kopecks = k;
        m.normalize();
    }
    else {
        is.setstate(ios::failbit);
    }

    return is;
}

int main() {
    setlocale(LC_ALL, "Russian");

```

```

cout << " Демонстрация работы класса Money \n\n";

// Создание объектов
cout << "1. Создание объектов:\n";
Money m1;
Money m2(150, 75);
Money m3(m2);

cout << "m1 = " << m1 << endl;
cout << "m2 = " << m2 << endl;
cout << "m3 (копия m2) = " << m3 << endl;

// Работа с сеттерами
cout << "\n2. Работа с сеттерами:\n";
m1.setRubles(10);
m1.setKopecks(50);
cout << "m1 после setRubles(10) и setKopecks(50): " << m1 << endl;

m1.setMoney(5, 150);
cout << "m1 после setMoney(5, 150): " << m1 << endl;

// Оператор присваивания
cout << "\n3. Оператор присваивания:\n";
Money m4;
m4 = m2;
cout << "m4 = m2 => m4 = " << m4 << endl;

// Операции сравнения
cout << "\n4. Операции сравнения:\n";
Money a(100, 50);
Money b(100, 50);
Money c(200, 0);

cout << "a = " << a << ", b = " << b << ", c = " << c << endl;
cout << "a == b: " << (a == b ? "true" : "false") << endl;
cout << "a != c: " << (a != c ? "true" : "false") << endl;

// Префиксный декремент
cout << "\n5. Префиксный декремент (--Money):\n";
Money d(10, 0);
cout << "d = " << d << endl;
cout << "--d = " << --d << endl;
cout << "d после = " << d << endl;

// Постфиксный декремент
cout << "\n6. Постфиксный декремент (Money--):\n";
Money e(5, 30);
cout << "e = " << e << endl;
cout << "e-- = " << e-- << endl;
cout << "e после = " << e << endl;

// Граничные случаи
cout << "\n7. Граничные случаи:\n";
Money zero(0, 0);
cout << "zero = " << zero << endl;
cout << "--zero = " << --zero << endl;

Money ruble(1, 0);
cout << "ruble = " << ruble << endl;
cout << "--ruble = " << --ruble << endl;

Money test(0, 99);
cout << "test = " << test << endl;
cout << "--test = " << --test << endl;

// Ввод с клавиатуры
cout << "\n8. Ввод с клавиатуры:\n";
cout << "Введите сумму (например, 123,45): ";

```

```

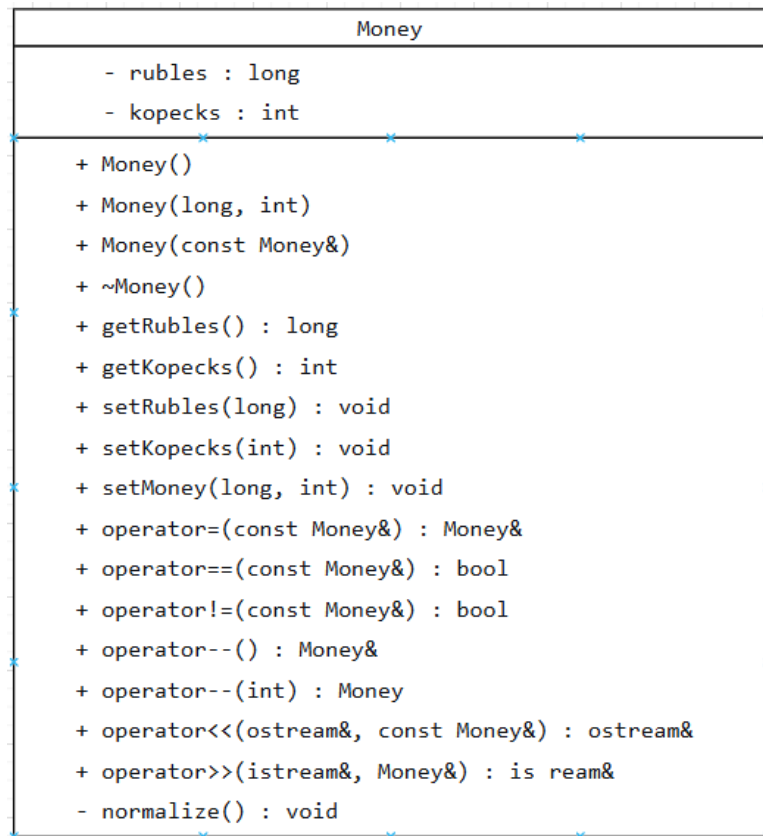
Money inputMoney;
cin >> inputMoney;
cout << "Вы ввели: " << inputMoney << endl;

cout << "\n Программа завершена \n";

return 0;
}

```

UML диаграмма



Вывод программы

Демонстрация работы класса Money

```
1. Создание объектов:
m1 = 0,00
m2 = 150,75
m3 (копия m2) = 150,75

2. Работа с сеттерами:
m1 после setRubles(10) и setKopecks(50): 10,50
m1 после setMoney(5, 150): 6,50

3. Оператор присваивания:
m4 = m2 => m4 = 150,75

4. Операции сравнения:
a = 100,50, b = 100,50, c = 200,00
a == b: true
a != c: true

5. Префиксный декремент (--Money):
d = 10,00
--d = 9,99
d после = 9,99

6. Постфиксный декремент (Money--):
e = 5,30
e-- = 5,30
e после = 5,29

7. Граничные случаи:
zero = 0,00
--zero = 0,00
ruble = 1,00
--ruble = 0,99
test = 0,99
--test = 0,98

8. Ввод с клавиатуры:
Введите сумму (например, 123,45):
```

Ответы на вопросы

1. Дружественные функции и классы используются для предоставления доступа к закрытым и защищённым членам класса извне, минуя открытый интерфейс. Это полезно при перегрузке операторов, особенно бинарных, где первый операнд не является объектом данного класса, а также при реализации тесного взаимодействия между классами.
2. Правила описания и особенности дружественных функций. Дружественная функция объявляется внутри класса с ключевым словом `friend`. Она не является членом класса и не имеет указателя `this`. Может быть объявлена в любой секции класса: `public`, `private` или `protected`. Её определение пишется вне класса как обычной функции. Дружественные классы: если класс А объявлен другом В, то А имеет доступ ко всем членам В. Дружба не наследуется и не транзитивна.
3. Перегрузить унарные операции можно двумя способами. Первый способ: методом класса без параметров, так как операндом является текущий объект

this. Второй способ: дружественной функцией с одним параметром - объектом.

4. Унарная функция-операция, определяемая внутри класса, должна иметь ноль явных параметров, так как один неявный операнд - это this.

5. Унарная функция-операция, определяемая вне класса, должна иметь один параметр - объект, над которым выполняется операция.

6. Бинарная функция-операция, определяемая внутри класса, должна иметь один явный параметр - второй операнд, так как первый операнд - это this.

7. Бинарная функция-операция, определяемая вне класса, должна иметь два параметра - левый и правый операнды.

8. Отличие перегрузки префиксных и постфиксных унарных операций в следующем. Префиксная форма `operator++` не имеет параметров и возвращает изменённый объект, обычно по ссылке. Постфиксная форма `operator++` имеет фиктивный параметр `int` и возвращает старое значение, обычно по значению, а затем изменяет объект.

9. Операцию присваивания можно перегрузить только как нестатический метод класса. Дружественная форма запрещена. Нужно избегать самоприсваивания и правильно управлять ресурсами, следуя правилу трёх или пяти.

10. Операция присваивания должна возвращать ссылку на текущий объект, то есть `return this`, чтобы поддерживать цепочки присваиваний вида `a = b = c`.

11. Операции ввода-вывода перегружаются только дружественными функциями или обычными функциями, но не методами, потому что левый операнд - это поток `ostream` для вывода или `istream` для ввода. Например, `friend ostream& operator<<(ostream& os, const MyClass& obj);` и `friend istream& operator>>(istream& is, MyClass& obj);`

12. Компилятор воспримет вызов как вызов метода класса `Student::operator++` с неявной передачей указателя `this` на объект `s`. Это префиксный инкремент, так как `operator++` определён внутри класса без параметров.

13. Компилятор воспримет вызов как вызов обычной дружественной функции `operator++` с передачей объекта `s` по ссылке. Поскольку функция объявлена как `friend Student& operator++(Student&)`, это префиксный инкремент.

14. Здесь возникает проблема приоритета операторов. Выражение `cout << a < b` будет интерпретировано как `(cout << a) < b`, потому что оператор `<<` имеет более высокий приоритет, чем `<`. Компилятор сначала вызовет `operator<<` для

cout и a, затем попытается сравнить результат вызова cout с b через operator<. Если тип возврата operator<< не поддерживает операцию меньше, возникнет ошибка. Правильно писать cout << (a < b). По сути вопроса: вызов оператора меньше как метода a.operator(b).

15. Аналогично предыдущему. Выражение cout << a > b будет интерпретировано как (cout << a) > b. Сначала вызывается operator<<, затем попытка сравнить результат потока с b через дружественную функцию operator>. Первым аргументом окажется не объект типа Person, а поток, поэтому возникнет ошибка. Если бы выражение было просто a > b, то компилятор вызвал бы дружественную функцию operator>(a, b).